JavaScript Part 2

endava

AGENDA

1. Scope & Closures
    - Lexical Scope
    - Closures
2. Prototypes & Inheritance
    - Prototypal Inheritance
    - ES6 Class Inheritance
3. Asynchronous JavaScript
    - Callbacks
    - Promises
    - Async / Await
4. Modules & Namespaces
5. Common JavaScript Patterns
6. Questions

1. Scope & Closures

LEXICAL SCOPE

- Lexical scope means a function can access variables from where it was defined, not where it is called.

Important Points

- Scope is determined at write time
- Inner functions access outer variables
- Based on execution context

```
function outer() {
  const message = "Hello";

  function inner() {
    console.log(message);
  }

  inner();
}

outer();

// Output: Hello
// Explanation: inner() can access message because
// scope is determined where the function is defined.
```

CLOSURES

- A closure is when a function remembers variables from its lexical scope after the outer function has finished.

Important Points

- Enables private variables
- Used in module pattern
- Can cause memory leaks if references persist

```
function outer() {
  let count = 0;

  return function inner() {
    count++;
    console.log(count);
  };
}

const counter = outer();
counter();
counter();

// Output: 1 2
// Explanation: inner() remembers count even after
// outer() has finished executing.
```

2. Prototypes & Inheritance

PROTOTYPAL INHERITANCE

Objects inherit properties and methods through the prototype chain.

Important Points

- Every object has a prototype
- Methods are shared (memory efficient)
- Foundation of JavaScript OOP

```javascript
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function () {
  console.log(this.name + " makes a sound");
};

const dog = new Animal("Dog");
dog.speak();

// Output: Dog makes a sound
// Explanation: speak() is shared through the prototype,
// not copied into each instance.
```

ES6 CLASS INHERITANCE

Classes provide cleaner syntax over prototypes.

Important Points

- extends creates inheritance
- super() calls parent constructor
- Still prototype-based internally

```
class Animal {
  speak() {
    return "Sound";
  }
}

class Dog extends Animal {
  speak() {
    return "Bark";
  }
}

const dog = new Dog();
console.log(dog.speak());
```

3. Asynchronous JavaScript

CALLBACK

A callback is a function passed to be executed later.

```
function greet(name, callback) {
  console.log("Hi " + name);
  callback();
}

greet("Juan", () => console.log("Done"));

// Output: Hi Juan
//         Done
// Explanation: A callback is a function passed as argument
// and executed later.
```

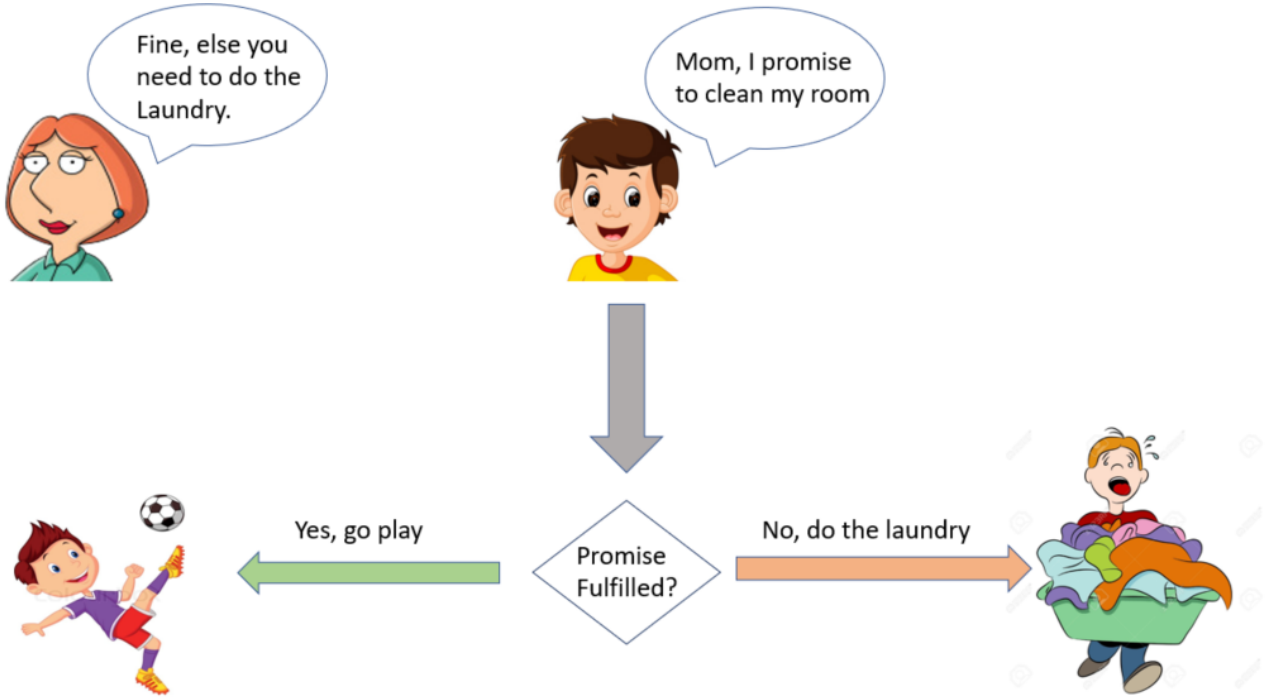
Important Points

- Simple async mechanism
- Can lead to callback hell
- Hard to maintain when nested

```
setTimeout(() => {
  console.log("Step 1");

  setTimeout(() => {
    console.log("Step 2");

    setTimeout(() => {
      console.log("Step 3");
    }, 1000);

  }, 1000);

}, 1000);
```

PROMISES

A Promise represents a future value.



PROMISES

Important Points

- States: Pending, Fulfilled, Rejected
- Supports chaining
- Cleaner than nested callbacks

```
const promise = new Promise((resolve) => {
  resolve("Success");
});

promise.then(result => console.log(result));

// Output: Success
// Explanation: A Promise represents a future value.
// .then() runs when the promise is fulfilled.
```

ASYNC/AWAIT

Async/await simplifies promise handling.

Important Points

- Must be inside async function
- Built on top of promises
- Improves readability

```
async function fetchData() {
  try {
    const response = await fetch("/api");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

4. Modules & Namespaces

MODULES

JavaScript

```javascript
export function add(x, y) {
  return x + y;
}

export function subtract(x, y) {
  return x - y;
}
```

```
const Try = () => {
  return (
    <>
      <h1>Try Page</h1>
      <AnotherNamedExport />
    </>
  );
};

export default Try;

export const AnotherNamedExport = () => {
  return <h2>Hi there</h2>;
};
```

```javascript
// names.js module
function lastname() { // private function
  return "Doe";
}

function firstname() { // private function
  return "Jane";
}

function firstnameLastname() { // public function
  return `${firstname()} ${lastname()}`;
}

function lastnameFirstname() { // public function
  return `${lastname()}, ${firstname()}`;
}

module.exports = { firstnameLastname, lastnameFirstname };
```

NAMESPACES (OLD PATTERN)

Grouping functionality inside objects.

Important Points

- Used before ES Modules
- Prevented global conflicts
- Mostly replaced today

```
const Utils = {
  sum(a, b) {
    return a + b;
  }
};
```

5. Common JavaScript Patterns

Common JS Patterns

- **Module Pattern:** Used to encapsulate logic and avoid polluting the global scope. Modern version = ES Modules (export / import).

```
const Counter = (function () {
  let count = 0; // private

  return {
    increment() {
      count++;
      console.log(count);
    },
    decrement() {
      count--;
      console.log(count);
    }
  };
})();

Counter.increment(); // 1
Counter.increment(); // 2
```

`count` is private and cannot be accessed directly.

Common JS Patterns

- **Factory Pattern**: Function that returns an object. No need for new. Very common in functional-style JS.

```
function createUser(name, role) {
  return {
    name,
    role,
    greet() {
      console.log(`Hi, I'm ${name}`);
    }
  };
}

const user1 = createUser("John", "admin");
user1.greet();
```

Common JS Patterns

- **Constructor Pattern**: Uses new to create instances. Modern equivalent = class

```
function User(name) {
  this.name = name;
}

User.prototype.greet = function () {
  console.log(`Hello ${this.name}`);
};

const user = new User("Alice");
user.greet();
```

Common JS Patterns

- **Singleton Pattern:** Ensures only one instance exists.

```
const Database = (function () {
  let instance;

  function createInstance() {
    return { connected: true };
  }

  return {
    getInstance() {
      if (!instance) {
        instance = createInstance()
      }
      return instance;
    }
  };
})();

const db1 = Database.getInstance();
const db2 = Database.getInstance();

console.log(db1 === db2); // true
```

Common JS Patterns

- **Observer Pattern**: One object notifies others when something changes.

Used in:

- Event systems
- React state updates
- RxJS
- Node's EventEmitter is based on this pattern.

```javascript
class Subject {
  constructor() {
    this.observers = [];
  }

  subscribe(fn) {
    this.observers.push(fn);
  }

  notify(data) {
    this.observers.forEach(fn => fn(data));
  }
}

const subject = new Subject();

subject.subscribe(data => console.log("Observer 1:", data));
subject.subscribe(data => console.log("Observer 2:", data));

subject.notify("Hello");
```

6. QUESTIONS

QUESTIONS

- **1)** Lexical scope is determined by?
  A) Execution time
  B) Where the function is defined
  C) The event loop
  D) The prototype chain

QUESTIONS

✅ **Answer:** B
➡️ **Explanation:** JavaScript uses lexical (static) scope. This means the scope of a function is determined by its position in the source code, not where or when it is executed.

- **2)** A closure allows:
  A) Private state
  B) Faster execution
  C) Multiple inheritance
  D) Threading

QUESTIONS

✅ **Answer:** A
➡️ **Explanation:** A closure allows a function to remember variables from its outer scope, even after the outer function has finished executing. This enables: Encapsulation, Data privacy and Private variables

- **3)** ES6 inheritance uses:
  A) inherit
  B) prototype keyword
  C) extends
  D) include

QUESTIONS

✅ **Answer:** C
➡️ **Explanation:** The extends keyword creates inheritance between classes.

- **4)** Promise states are?
  A) Start, End
  B) Ready, Done
  C) Pending, Fulfilled, Rejected
  D) True, False

QUESTIONS

✅ **Answer:** C
➡️ **Explanation:** Pending, Fulfilled, Rejected.

- **5)** Modules primarily prevent:
  A) Memory leaks
  B) Callback hell
  C) Global pollution
  D) Prototype conflicts

QUESTIONS

✅ **Answer:** C

➡️ **Explanation:** Before ES Modules, JavaScript code was loaded in the global scope. This caused naming conflicts, overwritten variables, hard-to-maintain code. Modules solve this by: creating file-level scope, exporting only what is needed, importing explicitly

6) What will be printed?

```
function test() {
  console.log(a);
  var a = 5;
}

test();
```

A) 5

B) Undefined

C) ReferenceError

D) null

QUESTIONS

✅ **Answer:** B
➡️ **Explanation:** var is hoisted to the top of the function. The value is undefined

7) What will be printed?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

A) 0 1 2
B) 3 3 3
C) undefined undefined undefined
D) 0 0 0

QUESTIONS

✅ **Answer:** B (3 3 3)
➡️ **Explanation:** var is function-scoped and the loop finishes first (i = 3), then callback is executed, all callbacks reference the SAME i, If we used “let”, it would print 0 1 2.

8) What will be printed?

```
function outer() {
  let count = 0;
  return function () {
    count++;
    console.log(count);
  };
}

const counter1 = outer();
const counter2 = outer();

counter1();
counter1();
counter2();
```

A) 1 2 3
B) 1 2 1
C) 1 1 1
D) 0 1 0

QUESTIONS

✅ **Answer:** B (1 2 1)
➡️ **Explanation:** Each call to outer() creates a new closure with its own count. Counter1 has own count and counter2 has a different count. Execution: counter1() → 1, counter1() → 2, counter2() → 1

9) What will be printed?

```
function Person(name) {
  this.name = name;
}

Person.prototype.sayHi = function () {
  return "Hi " + this.name;
};

const p1 = new Person("Juan");
const p2 = new Person("Ana");

console.log(p1.sayHi === p2.sayHi);
```

A) False
B) True
C) Undefined
D) Error

QUESTIONS

✅ **Answer:** B
➡️ **Explanation:** sayHi lives on the prototype, not inside each object. Both instances share the SAME function reference. If console.log(p1.sayHi() === p2.sayHi()); prints false

10) What does this print?

```
async function test() {
  console.log("A");
  await Promise.resolve();
  console.log("B");
}

console.log("C");
test();
console.log("D");
```

A) A C D B
B) C A B D
C) C A D B
D) A B C D

QUESTIONS

✅ **Answer:** C (C A D B)
➡️ **Explanation:** synchronous → runs immediately, Async → runs later. Execution order:

1) console.log("C") → C
2) test() runs
   - Logs A
   - hits await → pauses (microtask, "Pause here. Continue later.“, So test() stops at this line.)
3) console.log("D") → D
4) The awaited promise finishes (almost instantly, but still async). Runs console.log("B");

QUESTIONS

```
async function test() {
  console.log("A");
  await Promise.resolve();
  console.log("B");
}

console.log("C");
await test();
console.log("D");
```

THANKS